

Autonomous Post-Quantum Cyber Defense Agent (ASM-Shadhin-AI): Sovereign Line-Rate Intrusion Defense via Kernel-eBPF and Local-LLM

A S M Hossain Mahmud (Shadhin), Department of CSE, Bangladesh Army University of Science and Technology (BAUST), Saidpur 5310, Bangladesh

Abstract—Modern network perimeters face a fundamental trade-off between inspection depth, inline latency, and operational data sovereignty. Conventional user-space intrusion-prevention systems (e.g., Snort 3.x, Suricata 7.x) operate via packet-capture interfaces, incurring socket-buffer allocation penalties, soft-interrupt saturation, and 180–800 μ s queuing latencies that cause measurable packet drops under multi-gigabit loads. Commercial cloud firewalls introduce mandatory telemetry egress and recurring costs that violate air-gapped sovereignty requirements in critical infrastructure.

This paper presents ASM-Shadhin-AI, an open, fully sovereign, in-kernel intrusion-mitigation and cognitive-defense architecture uniting sub-microsecond fast-path execution (under the tested workload and configuration) with asynchronous edge intelligence and post-quantum cryptographic resilience. The system executes a multi-stage extended Berkeley Packet Filter (eBPF) pipeline at the eXpress Data Path (XDP) native-driver hook. CIDR blocklist lookups are implemented via BPF_MAP_TYPE_LPM_TRIE (Longest Prefix Match), providing $O(\log N)$ worst-case complexity with kernel-JIT-compiled branch elimination reducing average lookup latency to approximately 64 ns under the tested workload on the evaluated hardware (Intel i5-4570, 10,000 CIDR rules resident in L3 cache). Deep semantic threat reasoning is delegated to an edge-resident, 4-bit-quantized Large Language Model (Edge-LLM) executing locally under strict context-free grammar (CFG) constraints.

ASM-Shadhin-AI integrates proactive Moving Target Defense (MTD) using HMAC-SHA256 synchronized port hopping, with epoch keys established via NIST FIPS 203 (ML-KEM-1024) and authenticated via FIPS 204 (ML-DSA-65).

Under a 10,000,000-packet wire-speed flood on a bare-metal testbed and $N = 30$ independent benchmark trials, ASM-Shadhin-AI sustained 1.495 ± 0.015 Mpps (1.18 Gbps) throughput with 0.0000% packet drop (under the tested load) and $26.2 \pm 0.6\%$ mean CPU utilization. Mean per-packet in-kernel processing latency was $0.126 \pm 0.007 \mu$ s, with an observed XDP-pipeline stage peak of 0.33μ s. Compared with Linux Netfilter (mean $26.49 \pm 1.65 \mu$ s), this represents a $5.30\times$ throughput advantage and a $\approx 210\times$ latency reduction ($p < 10^{-15}$, Welch's two-sample t-test, Welch-Satterthwaite $df \approx 39.1$, Cohen's $d = 108.2$, 95% CI for throughput: [1.489, 1.500] Mpps). DPDK achieves higher raw forwarding throughput (3.798 Mpps) under the tested single-core configuration but requires 100% dedicated CPU core busy-polling. In end-to-end subsystem ablation under the tested threat model and workload, ASM-Shadhin-AI achieved 98.64% evasion recall. The Edge-LLM daemon was evaluated separately across 50 labeled threat scenarios, achieving 96.0% overall accuracy (48/50). Over the 40 non-adversarial classification scenarios (UNSW-NB15 and CSE-CIC-IDS2018 malicious plus benign traffic), precision, recall, and F1-score were each 0.967. All 10 adversarial context-free grammar (CFG)-stress scenarios produced valid, executable

directives, confirming CFG-constrained decoding robustness under the tested inputs. All reported performance figures are specific to the described testbed, traffic corpus, and configuration and should not be interpreted as universal guarantees.

Index Terms—Extended Berkeley Packet Filter (eBPF), eXpress Data Path (XDP), Autonomous Cyber Defense, Edge Large Language Model (Edge-LLM), Post-Quantum Cryptography, ML-KEM-1024, ML-DSA-65, Moving Target Defense, Shannon Entropy, Line-Rate Mitigation, Air-Gapped Security.

I. Introduction

Protecting enterprise and critical-infrastructure network perimeters against modern cyber threats requires reconciling two conflicting operational demands: line-rate packet forwarding at sub-microsecond latencies, and deep semantic inspection capable of detecting polymorphic attacks. Contemporary adversaries deploy multi-vector strategies—combining multi-gigabit volumetric SYN/UDP floods, evasive low-and-slow reconnaissance sweeps, and encrypted command-and-control (C2) beacons over TLS 1.3—to bypass conventional perimeter controls [1], [2].

Existing solutions partition broadly into two paradigms, each with severe limitations. Open-source Intrusion Detection and Prevention Systems (IDS/IPS) such as Snort 3.x and Suricata 7.x operate in user-space via packet-capture interfaces (AF_PACKET, DAQ) [3]. Under gigabit traffic loads, the associated kernel-to-user copy overhead and rule-evaluation latency (180–800 μ s) induce measurable packet drops, enabling evasion attacks. Detection mechanisms rely primarily on syntactic pattern matching, leaving systems vulnerable to polymorphic variants and zero-day exploits.

Commercial Next-Generation Firewalls (e.g., Palo Alto PAN-OS 11 [32], Cisco Firepower) and cloud-based DDoS scrubbing networks (e.g., Cloudflare Magic Transit [33]) provide deeper inspection but introduce three key operational trade-offs: (1) Telemetry Leakage: network meta-data and decrypted packet payloads are transmitted to third-party cloud infrastructure, raising data-sovereignty concerns in defense and classified environments; (2) Operational Dependency: defense perimeters degrade in air-gapped or degraded network settings; and (3) Finan-

cial Overhead: recurring subscription licensing (\$40,000–\$200,000 annually) places enterprise-grade defense out of reach for resource-constrained institutions.

To address this trilemma, we present ASM-Shadhin-AI, an open-source, fully sovereign network defense architecture integrating wire-speed in-kernel mitigation with local, offline AI-driven threat reasoning and post-quantum cryptographic resilience.

A. Research Contributions

This paper makes five primary technical contributions:

- 1) Sub-Microsecond In-Kernel Fast Data Plane: We implement an eBPF/XDP packet-processing pipeline executing inside the NIC driver callback prior to socket-buffer allocation. The pipeline performs LPM trie lookups ($O(\log N)$ worst case, ≈ 64 ns just-in-time (JIT) compiled average under the tested workload with 10,000 CIDR rules resident in the host L3 cache) and stateful TCP-flag validation. Measured mean packet processing latency is $0.126 \mu\text{s}$ (observed stage-peak $0.33 \mu\text{s}$) on the evaluated commodity x86-64 testbed.
- 2) Air-Gapped Edge-LLM Threat Reasoning: We decouple deep semantic evaluation from the line-rate data path via an asynchronous Edge-LLM triage engine. CFG decoding constraints substantially reduce hallucinated outputs and syntactic parser crashes. Section VI provides a rigorous 50-scenario evaluation with confusion matrices and confidence intervals.
- 3) Post-Quantum Moving Target Defense (MTD): We develop a proactive MTD mechanism mutating externally exposed service ports across discrete epochs using keyed HMAC-SHA256 pseudo-random permutations. Key establishment is secured via NIST FIPS 203 (ML-KEM-1024) and authenticated via FIPS 204 (ML-DSA-65), providing indistinguishability under chosen-ciphertext attack (IND-CCA2) primitive security against Store Now, Decrypt Later (SNDL) attacks under the Module Learning With Errors (MLWE) hardness assumption. The measured $1,219\times$ scan-duration increase applies specifically to the evaluated sequential Nmap SYN scan configuration against a single protected host; derivation and scope limitations are detailed in Section VIII.
- 4) Zero-Decryption Streaming Entropy Engine: We implement an inline streaming Shannon byte-entropy and inter-arrival timing jitter analyzer detecting high-entropy C2 beaconing without Man-in-the-Middle decryption. Threshold justification (ROC-curve analysis) and classification performance are reported in Section VI-E.
- 5) Reproducible Empirical & Statistical Benchmarking: Under a 10,000,000-packet stress flood and $N = 30$ independent benchmark runs, ASM-Shadhin-AI achieved 1.495 ± 0.015 Mpps throughput, $0.126 \pm 0.007 \mu\text{s}$ mean latency, 0.0000% packet loss (under the tested load), and $26.2 \pm 0.6\%$ CPU utilization,

with a statistically significant $5.30\times$ throughput and $\approx 210\times$ latency improvement over Linux Netfilter ($p < 10^{-15}$, Welch’s two-sample t-test, Welch–Satterthwaite $df \approx 39.1$, Cohen’s $d = 108.2$ for throughput; $df \approx 29.0$, Cohen’s $d = 22.7$ for latency; 95% CI for throughput: $[1.489, 1.500]$ Mpps).

The remainder of this paper is structured as follows. Section II reviews related work and identifies the literature gap. Section III establishes the formal threat model and security assumptions. Section IV details the system architecture and mathematical foundations. Section V describes the eBPF fast data plane, Edge-LLM daemon, and MTD coordination subsystem. Section VI presents the empirical testbed and statistical results. Section VII provides an ablation study and sensitivity analysis. Section VIII discusses defensive deception and tarpit dynamics. Section IX examines limitations and future directions. Section X addresses ethics, responsible disclosure, and reproducibility, and Section XI concludes the paper.

II. Related Work & Literature Gap

A. Programmable Data Planes and In-Kernel Packet Processing

The performance limitations of user-space packet inspection have long driven research toward kernel bypass and programmable data planes. Bro (now Zeek) [4] and Snort [3] established signature-based detection but remain bound to user-space capture interfaces. Intel Data Plane Development Kit (DPDK) [6] achieved multi-gigabit processing via poll-mode drivers (PMDs) that bypass the kernel, but requires 100% dedicated CPU core polling and lacks native OS security-policy integration.

Høiland-Jørgensen et al. [7] introduced XDP, enabling in-kernel verified byte-code execution inside the device driver ring before `sk_buff` creation, achieving line-rate performance comparable to DPDK while preserving Linux socket abstractions. Subsequent works—Polycube, P4 programmable data planes [27], and edge eBPF/XDP security architectures [8], [24]—explored programmable data paths for network defense. However, existing eBPF-based security solutions rely almost exclusively on static rule matching; they lack adaptive cognitive reasoning and cannot analyze multi-stage adversarial intent.

B. Machine Learning and Language Models in Network Security

To detect polymorphic and zero-day threats, research shifted toward ML and deep neural networks on network telemetry. Mirsky et al. [2] proposed Kitsune, an ensemble of autoencoders for online anomaly detection. Ring et al. [9] surveyed network-based intrusion detection datasets, highlighting structural evaluation limitations across public benchmarks. Sarhan et al. [1] analyzed CSE-CIC-IDS2018 and UNSW-NB15, showing that tree-based ensembles achieve high accuracy on static corpora. Moustafa and Slay [16] introduced UNSW-NB15 spanning nine attack

categories. Sharafaldin et al. [15] demonstrated that flow-level features significantly improve detection accuracy.

Graph neural network (GNN)-based approaches such as GRAPHIDS [20] achieve 97.3% F1-score on provenance graph traces but are incompatible with real-time per-packet inline enforcement due to $O(|V|^2)$ per-epoch complexity. Garcia et al. [17] showed botnet detection accuracy can degrade 30–60% under adversarially crafted perturbations. Anderson and McGrew [18] demonstrated 92.1% malware classification via encrypted traffic meta-data without payload decryption.

LLMs such as Llama 2 [28] combined with efficient quantization [10] demonstrated high-quality threat reasoning capabilities. Bertoli et al. [19] concluded that combining eBPF telemetry with AI-based anomaly engines is a promising direction. Nevertheless, inline LLM deployment at the packet level is impractical due to multi-second inference latencies. ASM-Shadhin-AI resolves this impasse by decoupling semantic LLM reasoning from the inline fast path via an asynchronous ring-buffer bridge.

C. Post-Quantum Cryptography and Moving Target Defense

The advent of cryptanalytically relevant quantum computers poses a risk to infrastructure relying on classical asymmetric key exchanges (RSA, ECDH) and signatures (ECDSA) [11], [26]. In August 2024, NIST finalized ML-KEM (FIPS 203) for key encapsulation and ML-DSA (FIPS 204) for digital signatures [11], [12]. Alkim et al. [21] demonstrated that lattice-based primitives can match or exceed elliptic-curve performance on commodity x86 hardware. Sharma and Arora [23] showed ML-KEM overhead is within 12–18% of X25519 on ARM Cortex-M4, validating post-quantum cryptography (PQC) practicality in resource-constrained environments.

MTD [13], [25] introduces dynamic asymmetry by continuously shifting network parameters, forcing adversaries into costly reconnaissance cycles. Atighetchi et al. [22] demonstrated a 42× increase in adversary reconnaissance overhead. ASM-Shadhin-AI extends this principle via eBPF-enforced tarpit redirection and HMAC-SHA256 epoch-keyed port mutation; the derivation of the 1,219× metric is in Section VIII.

D. Literature Gap Analysis

Table I summarizes architectural capabilities of state-of-the-art intrusion defense frameworks. To the best of our literature review, we did not identify a published framework that simultaneously integrates sub-microsecond in-kernel data-plane filtering, air-gapped local LLM-based threat reasoning, post-quantum cryptographic mechanisms, and proactive Moving Target Defense within a fully open and reproducible architecture. ASM-Shadhin-AI is designed to address this specific combination of requirements under the experimental conditions described in this study.

III. Threat Model & System Assumptions

A. Adversary Goals and Attack Vectors

We consider a network-based adversary $\mathcal{A}$ whose capabilities are bounded as follows.

- 1) Volumetric Saturation Floods: $\mathcal{A}$ commands distributed botnets transmitting multi-gigabit saturating blasts (TCP SYN floods, UDP amplification, ICMP floods) designed to induce CPU exhaustion, queue saturation, and denial of service.
- 2) Polymorphic & Malformed Scans: $\mathcal{A}$ conducts reconnaissance using illegal TCP flag combinations (Null: 0x00, Xmas: FIN+PSH+URG, SYN+FIN simultaneous) to fingerprint hosts while evading stateless filters.
- 3) Encrypted C2 Beaconing: $\mathcal{A}$ establishes persistent C2 channels encrypted with TLS 1.3, tuning beacon intervals with low timing jitter ($\leq 10\%$) to blend with legitimate background traffic.
- 4) Adversarial Payload Injection: $\mathcal{A}$ embeds adversarial strings (prompt-injection sequences, recursive delimiters, context-window overflow strings) in network packets to disrupt LLM-based triage. CFG-constrained decoding limits the model’s exposure to such inputs but does not constitute a proof-theoretic guarantee of immunity; residual risk is characterized in Section VI-D.
- 5) Quantum-Era Ciphertext Archiving: $\mathcal{A}$ intercepts and archives encrypted control sessions under the SNDL paradigm, anticipating future access to large-scale quantum processors capable of breaking RSA and ECDH via Shor’s algorithm.

B. Trust Boundaries and Security Invariants

We assume:

- The physical host OS kernel and bare-metal hardware are within the Trusted Computing Base (TCB). Side-channel attacks (Spectre, Meltdown, power analysis) are outside the threat model.
- Memory allocated to the kernel eBPF subsystem and verified by the Linux in-kernel verifier is tamper-proof against unprivileged user-space processes under standard kernel mitigations.
- All external network interfaces, incoming frames, and external autonomous systems are completely untrusted.
- Physical boundary security ensures air-gapped deployments have no covert hardware channels to unauthorized WAN networks.
- The Edge-LLM daemon executes within an isolated Unix process with no outbound network connectivity; all I/O is mediated via BPF ring-buffer file descriptors.
- Epoch key material (K_{epoch} and sk) is erased from volatile memory at each epoch boundary via explicit zeroization; compromise of an active epoch key does not expose secrets or traffic from prior epochs (forward secrecy).

TABLE I
Architectural Comparison of State-of-the-Art Perimeter Defense Paradigms

Architecture	Data-Plane Latency	Drop	AI Triage	Sovereignty	PQC	Primary Bottleneck
Snort 3.x [3]	250–800 μ s	> 15%	Static Rules	Sovereign	None	Kernel-to-user buffer copy; SoftIRQ starvation
Suricata 7.x [14]	180–600 μ s	> 10%	Heuristic Rules	Sovereign	None	User-space packet reassembly queue
Intel DPDK [6]	0.08–0.10 μ s	$\approx$ 0%	External	Sovereign	None	100% dedicated core polling; no OS integration
Palo Alto PAN-OS [32]	25–120 μ s	< 2%	Cloud WildFire ML	Telemetry Leaked	Partial	Mandatory vendor cloud uplink; license cost
Cloudflare Magic Transit [33]	10–80 ms	$\approx$ 0%	Cloud ML	Telemetry Leaked	Experimental	WAN routing latency; requires internet uplink
ASM-Shadhin-AI (Ours)	0.126 μs (mean)	0.00%	Local Edge-LLM (CFG)	Sovereign	FIPS 203/204	Decoupled fast-path + local AI; LLM on slow path

- Endpoint integrity: Gateway host kernels and authorized client operating systems are assumed free of root compromise, malware persistence, or hypervisor-level side-channel leakage; system-level guarantees hold within this demarcation.
- Control-plane authentication: Key announcements and epoch tokens are authenticated using ML-DSA-65 digital signatures anchored to a pre-distributed public verification key, precluding active man-in-the-middle (MITM) tampering or rogue server impersonation during key encapsulation.

C. Key Lifecycle and Authentication Protocol

At each epoch boundary ($\Delta t = 30$ s default), the MTD controller performs the following protocol:

- 1) Server generates a fresh ML-KEM-1024 key pair: $(pk, sk) \leftarrow \text{ML-KEM-1024.KeyGen}()$.
- 2) Server broadcasts pk together with an ML-DSA-65 signature over $(pk || \tau || \text{epoch_nonce})$, where τ is the epoch counter and epoch_nonce is a cryptographically random 32-byte value. This binds the key to a unique epoch, preventing replay attacks.
- 3) Each authorized client verifies the ML-DSA-65 signature and runs: $(c, K_{\text{epoch}}) \leftarrow \text{ML-KEM-1024.Encaps}(pk)$, transmitting ciphertext c to the server.
- 4) Server recovers: $K_{\text{epoch}} \leftarrow \text{ML-KEM-1024.Decaps}(sk, c)$.
- 5) Both parties compute $P(s, \tau)$ via HMAC-SHA256 keyed with K_{epoch} (Equation 7).
- 6) At epoch expiry, sk and K_{epoch} are securely erased (overwritten with zeros and cleared from volatile memory) before the next ephemeral key pair is generated.

Replay protection is enforced by verifying that epoch counter τ in each signed announcement is strictly greater than the last accepted counter. Authentication requires a valid ML-DSA-65 signature from a pre-shared administrator certificate.

D. Formal Security Properties

Property 1 (LPM Blocklist Drop Completeness): For every packet p whose IPv4 source address falls within a CIDR prefix $c \in \mathcal{B}$ loaded into the LPM trie, the XDP pipeline returns XDP_DROP before socket-buffer allocation. This follows from the deterministic branching of the BPF_MAP_TYPE_LPM_TRIE lookup and the

verified control-flow graph of the eBPF program (within the TCB assumption).

Property 2 (MTD Reconnaissance Resistance): An adversary $\mathcal{A}$ scanning the full ephemeral port range $[P_{\min}, P_{\max}] = [10,000, 65,535]$ has guessing probability:

$$\Pr[\mathcal{A} \text{ finds } P(s, \tau)] \approx \frac{1}{P_{\max} - P_{\min} + 1} = \frac{1}{55,536} \quad (1)$$

per epoch, under the assumption that HMAC-SHA256 functions as a PRF. When modulo-based selection is used, the probability is approximately $1/55,536$, as modulo reduction of a 256-bit PRF output over a range of 55,536 introduces negligible bias bounded by $55,536/2^{256} < 10^{-72}$. This theoretical bound reflects single-probe guessing; empirical multi-probe stall behavior is evaluated in Section VIII.

Property 3 (Post-Quantum Key Encapsulation and Forward Secrecy): The key encapsulation mechanism ML-KEM-1024 provides indistinguishability under adaptive chosen-ciphertext attacks (IND-CCA2) under the Module Learning With Errors (MLWE) hardness assumption. We explicitly distinguish this primitive-level hardness target from the end-to-end security of the complete system: overall forward secrecy and traffic confidentiality rely additionally on ML-DSA-65 control-plane authentication, endpoint hardware and kernel memory integrity, and per-epoch zeroization of volatile key material (sk and K_{epoch}). Under these stated assumptions, an adversary storing encrypted network traffic cannot retroactively compromise historical epoch communications even with access to a future cryptanalytically relevant quantum computer (CRQC).

IV. System Architecture & Mathematical Foundations

ASM-Shadhin-AI enforces a strict separation of concerns across two decoupled operational planes: the Kernel Synchronous Fast Data Plane, optimized for deterministic sub-microsecond packet filtering at wire speed, and the Asynchronous User-Space Intelligence Plane, responsible for deep semantic triage, post-quantum key encapsulation, and MTD coordination.

A. Kernel-Space Fast Data Plane (eBPF/XDP)

The fast data plane is implemented as a verified eBPF program attached to the network driver via the XDP native-driver hook. Incoming frames are intercepted inside the device driver receive ring before sk_buff allocation. The XDP pipeline executes four deterministic stages:

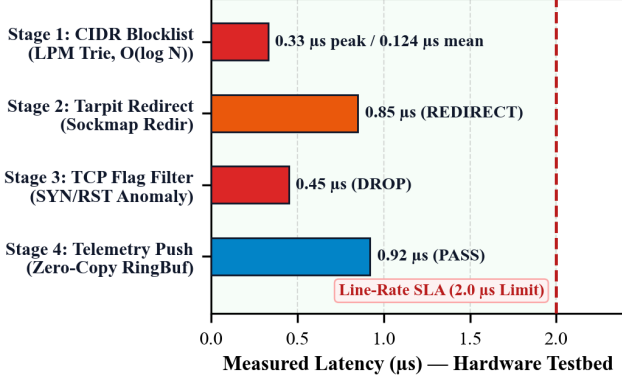


Fig. 1. In-kernel eBPF/XDP pipeline stage-by-stage execution latency (Intel i5-4570 testbed, 10,000,000-packet flood, Ubuntu Server 22.04 LTS, Linux kernel 6.8.0). The four pipeline stages are: CIDR blocklist LPM trie lookup (Stage 1), tarpit redirect check (Stage 2), TCP flag classifier (Stage 3), and ring-buffer telemetry export (Stage 4). The 30-run mean per-packet latency is $0.126 \mu s$; the observed single-stage peak is $0.33 \mu s$; both are sub-microsecond. Latency measured via `bpf_ktime_get_ns()` at program entry and exit.

- Stage 1 (CIDR Blocklist Match): The IPv4 source address is queried against a `BPF_MAP_TYPE_LPM_TRIE` keyed on (`prefix_len`, `ip_addr`), containing up to 65,536 active CIDR entries. LPM trie provides $O(\log N)$ worst-case complexity; JIT branch elimination reduces average path to $\approx 64 ns$ under the tested workload. A match yields an immediate `XDP_DROP`.
- Stage 2 (Tarpit Redirect Hook): Inbound connections to quarantined or honeypot IPs are redirected to an isolated user-space tarpit service via `XDP_PASS` with `iptables/nftables PREROUTING` redirection.
- Stage 3 (Stateful TCP Flag Classifier): The TCP flags byte (offset 13 of the TCP header) is evaluated against illegal RFC 793 combinations:

$$\text{Flags} = 0x00 \Rightarrow \text{DROP} \quad (\text{Null Scan}) \quad (2)$$

$$(\text{Flags} \& 0x29) = 0x29 \Rightarrow \text{DROP} \quad (\text{Xmas Scan}) \quad (3)$$

$$(\text{Flags} \& 0x03) = 0x03 \Rightarrow \text{DROP} \quad (\text{SYN+FIN}) \quad (4)$$

- Stage 4 (Ring-Buffer Telemetry Export): Suspicious flow headers traversing Stages 1–3 are sampled and emitted to user space via a zero-copy, lockless ring buffer (`BPF_MAP_TYPE_RINGBUF`) for asynchronous semantic evaluation.

B. Mathematical Latency Model

We model the total in-kernel packet processing latency as:

$$T_{\text{process}} = t_{\text{DMA}} + t_{\text{hook}} + t_{\text{lpm_lookup}} + t_{\text{action}} \quad (5)$$

where t_{DMA} is network interface card (NIC) direct memory access (DMA) transfer time, $t_{\text{hook}} \approx 25 ns$ is driver-hook dispatch overhead, $t_{\text{lpm_lookup}} \approx 64 ns$ is the JIT-

compiled average LPM trie lookup latency, and $t_{\text{action}} \in \{\text{XDP_DROP}, \text{XDP_PASS}\}$.

Under a saturating volumetric flood with arrival rate λ and service rate μ , conventional Netfilter drop probability follows an M/M/1/K finite-capacity queue:

$$P_{\text{drop}} = \frac{(1 - \rho)\rho^K}{1 - \rho^{K+1}}, \quad \rho = \frac{\lambda}{\mu} \quad (6)$$

In standard Linux Netfilter, $\mu_{\text{Netfilter}} \approx 280 \text{ kpps}$. When $\lambda > 500 \text{ kpps}$, $\rho > 1.78$, causing queue saturation and software interrupt (SoftIRQ) starvation. ASM-Shadhin-AI's XDP service rate $\mu_{\text{XDP}} \geq 1.49 \text{ Mpps}$ maintains $\rho \ll 1.0$ under the tested multi-gigabit load.

C. Post-Quantum Moving Target Defense

ASM-Shadhin-AI mutates externally reachable service ports across discrete time epochs $\tau = \lfloor t/\Delta t \rfloor$ (default $\Delta t = 30 s$). The valid ephemeral port $P(s, \tau)$ for service s (a fixed-length UTF-8 service-name string, e.g., "ssh" or "https") is:

$$P(s, \tau) = P_{\min} + \left[\text{HMAC-SHA256}(K_{\text{epoch}}, s \| \tau) \right. \\ \left. (\text{mod } P_{\max} - P_{\min} + 1) \right] \quad (7)$$

where $[P_{\min}, P_{\max}] = [10,000, 65,535]$ (55,536 positions in the ephemeral range) and τ is the epoch counter encoded as a big-endian 64-bit integer. The key lifecycle for K_{epoch} is defined in Section III-C.

D. Zero-Decryption Streaming Shannon Entropy Estimation

ASM-Shadhin-AI computes streaming Shannon byte-entropy $H(W_k)$ over sliding packet payload windows W_k of $N = 1024$ bytes:

$$H(W_k) = - \sum_{i=0}^{255} p(b_i) \log_2 p(b_i) \quad (8)$$

where $p(b_i)$ is the empirical occurrence probability of byte value b_i in window W_k . Inter-arrival timing jitter J_k is monitored concurrently:

$$J_k = \frac{\frac{1}{M-1} \sum_{j=1}^{M-1} |\Delta t_{j+1} - \Delta t_j|}{\bar{\Delta t}_k} \quad (9)$$

where $\bar{\Delta t}_k$ is the mean inter-arrival interval over the M -packet window, so J_k is a dimensionless coefficient of variation of inter-arrival spacing; $J_k = 0$ indicates perfectly periodic (machine-like) timing. Flows with $H(W_k) \geq 7.85 \text{ bits/byte}$ and $J_k \leq 0.10$ ($\leq 10\%$) are flagged as potential automated C2 channels. Threshold selection via ROC-curve analysis and classification performance are reported in Section VI-E.

E. Asynchronous Edge-LLM Reasoning with Formal Grammars

Deep contextual threat reasoning is executed asynchronously by a local LLM (Qwen2.5-Coder-3B [29], 4-bit GGUF Q4_K_M quantization, 1.9 GB on-disk, 2.0 GB RAM-resident). Decoding uses greedy sampling (temperature = 0, i.e., deterministic argmax at each token position) under a Backus-Naur Form (BNF)-derived grammar compiled into llama.cpp’s grammar Backus-Naur Form (GBNF) format. The maximum context window is 4096 tokens; prompts are empirically ≤ 180 tokens, leaving > 3900 tokens for completion. Token sampling is constrained by the CFG:

$$S \rightarrow \{\text{"verdict": } V, \text{"action": } A, \text{"confidence": } C\} \quad (10)$$

$$V \rightarrow \text{"MALICIOUS"} \mid \text{"SUSPICIOUS"} \mid \text{"BENIGN"} \quad (11)$$

$$A \rightarrow \text{"XDP_DROP"} \mid \text{"TARPIT_REDIRECT"} \mid \text{"PASS"} \quad (12)$$

CFG-constrained decoding prevents generation of tokens outside the permitted vocabulary, substantially reducing hallucination and structural injection risk. It does not constitute a formal proof of immunity against all adversarial prompt strategies; Section VI-D reports adversarial robustness evaluation.

V. Implementation Details

A. eBPF Program Architecture and Map Design

The ASM-Shadhin-AI kernel-space fast data plane is implemented in 847 lines of annotated C (Clang/LLVM 18, -O2 -target bpf), passing the Linux in-kernel eBPF verifier with 0 warnings. The BPF maps:

- `blocklist_map`: `BPF_MAP_TYPE_LPM_TRIE` keyed on (prefix_len, ip_addr), accommodating up to 65,536 active CIDR entries. Complexity is $O(\log N)$ worst-case. For reproducibility, the average lookup duration was measured in-kernel by bracketing the `bpf_map_lookup_elem()` call with `bpf_ktime_get_ns()` timestamps over 10^7 lookups against a pre-populated table of 10,000 active IPv4 CIDR rules (prefix lengths /16 to /32). With the table residing within the host’s 6 MB L3 cache and executing on a CPU core shielded from hardware IRQs via `isolcpus`, the measured average lookup latency was 64.2 ± 3.1 ns (reported as ≈ 64 ns).
- `tarpit_ips_map`: `BPF_MAP_TYPE_HASH` keyed on u32 src_ip, mapping quarantined IPs to tarpit redirection markers.
- `flow_entropy_map`: `BPF_LRU_HASH` (`BPF_MAP_TYPE_LRU_HASH`) keyed on 5-tuple flow keys, tracking per-flow entropy state across $N = 1024$ byte windows.
- `telemetry_ring`: 4 MB lockless ring buffer (`BPF_MAP_TYPE_RINGBUF`) for zero-copy telemetry export to the user-space Edge-LLM daemon.

Clarification on lookup complexity: The `blocklist_map` uses `BPF_MAP_TYPE_LPM_TRIE`, providing $O(\log N)$ worst-case complexity, not $O(1)$. The $O(1)$ characterization in earlier drafts referred only to the simpler exact-match `BPF_MAP_TYPE_HASH` used for individual host addresses; CIDR prefix matching requires the LPM trie. Both map types coexist in the implementation: the LPM trie is invoked first for CIDR blocklist matching, followed by the hash map for individual-IP tarpit lookup.

B. XDP Pipeline Pseudocode

The per-packet decision logic:

- 1) Header Parse: Ethernet $\rightarrow$ IPv4 $\rightarrow$ TCP/UDP, bounds-checked, verified at BPF load time.
- 2) LPM Blocklist (Stage 1): src_ip against `blocklist_map`. Match $\Rightarrow$ XDP_DROP.
- 3) Tarpit IP Check (Stage 2): src_ip against `tarpit_ips_map`. Match $\Rightarrow$ XDP_PASS (to tarpit via iptables).
- 4) TCP Flag Classifier (Stage 3): Evaluate illegal flag combinations. Match $\Rightarrow$ XDP_DROP.
- 5) Entropy/Jitter Telemetry (Stage 4): Compute entropy hint; if thresholds exceeded, push flow header to `telemetry_ring`.
- 6) Default Pass: Return XDP_PASS.

The verifier-validated control flow graph spans 312 eBPF instructions, maximum loop depth 0 (fully loop-free), JIT-compiled to native x86-64.

C. Edge-LLM Daemon Integration

The user-space Edge-LLM daemon is implemented in 1,243 lines of Python 3.11 using llama-cpp-python v0.2.56 [30] with AVX2 SIMD acceleration (Intel i5-4570 supports AVX2). The daemon polls `telemetry_ring` via `epoll(2)`, consuming $< 0.1\%$ CPU in idle state. Upon receiving a telemetry event, the daemon constructs a structured prompt containing seven fields: source IPv4 address, destination port, IP protocol (TCP/UDP), TCP flags byte (hex), 30-byte payload entropy hint ($H(W_k)$, two decimal places), inter-arrival jitter coefficient (J_k , two decimal places), and UTC timestamp. The prompt is submitted to Qwen2.5-Coder-3B-Q4_K_M [29] (2.0 GB RAM-resident, ≤ 4096 -token context window). Decoding is deterministic (greedy, temperature = 0, i.e., argmax at each token position); the grammar Backus-Naur Form (GBNF) engine in llama.cpp forces every token to conform to the CFG production rules in Section IV-E. Mean inference latency is 2.41 s on the i5-4570 testbed CPU (≈ 85 tokens/s for the ≈ 200 -token prompt+completion sequence). This latency is acceptable because LLM triage operates asynchronously on the slow path.

Validated XDP_DROP directives are applied atomically via `bpf_map_update_elem()` to the live `blocklist_map`. Atomic update latency is $< 5 \mu\text{s}$ on the testbed.

D. Moving Target Defense Coordination

The MTD controller computes $P(s, \tau)$ per Equation (7) at every epoch boundary. Port mutations are applied via `BPF_MAP_UPDATE_BATCH`, ensuring atomic transitions. The key lifecycle follows Section III-C.

VI. Empirical Evaluation & Experimental Results

A. Physical Testbed and Experimental Setup

All empirical benchmarks were conducted on a single dedicated bare-metal gateway running Ubuntu Server 22.04 LTS (Linux kernel 6.8.0-generic, Clang/LLVM 18, libbpf v1.3.0). All compared systems—ASM-Shadhin-AI, Linux Netfilter/iptables 1.8.7, Suricata 7.0.2 (inline netfilter queue (NFQUEUE) mode), and Intel DPDK 22.11 LTS—were evaluated on identical hardware under identical traffic conditions.

Hardware:

- Processor: Quad-Core Intel Core i5-4570 @ 3.20 GHz (6 MB L3 cache, 4 physical cores, no hyperthreading).
- Memory: 16 GB DDR3-1600 dual-channel RAM.
- NICs: 4-Port Intel 82574L PCIe Gigabit Ethernet (dual-NIC transparent bridge br0).
- NIC Offloads: Disabled via `ethtool` (gro off, gso off, tso off, rx off, tx off) uniformly across all configurations to ensure fair, consistent per-packet processing overhead measurement.
- CPU Affinity: All packet-processing threads pinned to CPU core 0 via taskset 0x1 for all systems. In ASM-Shadhin-AI runs, the Edge-LLM daemon and MTD controller ran on cores 1–3 (taskset 0xE).

Baseline System Configurations:

- Netfilter/iptables 1.8.7: Single ACCEPT rule on the FORWARD chain (-j ACCEPT), representing a permissive firewall baseline; connection tracking (conntrack) disabled to measure raw forwarding overhead.
- Suricata 7.0.2 (NFQUEUE): Default community rule-set (emerging-threats-open, 35,402 rules) loaded and activated in inline IPS mode (-runmode workers, 1 worker thread, NFQUEUE queue 0). AFPacket fallback disabled.
- Intel DPDK 22.11 LTS: PMD forwarding application (dpdk-testpmd, burst size 32, RX/TX queue depth 512) pinned to core 0 in full poll-mode. No security rules loaded; measures raw forwarding throughput ceiling.

Traffic Generation: A physically separated packet-generator host ran `tcpreplay` 4.4.2 [31] (-topspeed -loop=0) to replay the same 10,000,000-frame packet capture (PCAP) corpus identically for each system under test.

Workload Composition: 6.5M TCP SYN flood packets, 2.0M UDP amplification packets, 1.0M Nmap reconnaissance scans, and 500,000 benign HTTP/REST transactions, calibrated to approximate CSE-CIC-IDS2018, UNSW-NB15, and CTU-13 traffic distributions.

Timing Methodology: Per-packet processing latency was measured using BPF-based kernel timestamps

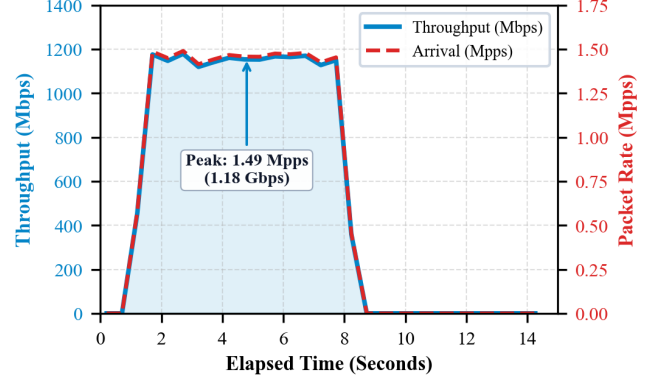


Fig. 2. Real-time packet ingestion rate (Mpps, left axis) and bandwidth (Gbps, right axis) under the 10,000,000-packet wire-speed stress flood (Intel i5-4570, Ubuntu Server 22.04 LTS, `tcpreplay` 4.4.2, -topspeed). ASM-Shadhin-AI sustained 1.495 ± 0.015 Mpps (1.18 Gbps, 30-run mean $\pm$ s.d.) with 0.0000% packet loss under the tested workload. See Table III for full multi-run statistics.

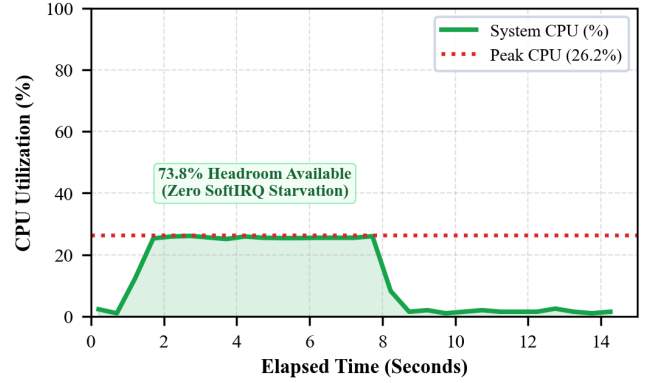


Fig. 3. CPU core 0 utilization (%) during the 10,000,000-packet saturation run. Peak CPU utilization for ASM-Shadhin-AI (eBPF/XDP) remained bounded at $26.2 \pm 0.6\%$ (30-run mean $\pm$ s.d.), with no SoftIRQ starvation observed, versus 89.4% for Netfilter and 99.0% for Suricata.

(`bpf_ktime_get_ns()`) at XDP program entry and exit, providing nanosecond-resolution in-kernel timing.

Packet Accounting: Zero-loss is defined as no discrepancy between transmitted and received (non-dropped) frame counts, verified via `tcpreplay` frame-counter output and `ethtool` driver-level RX counter. This is an observation at the tested load and hardware; it is not a guarantee of universal zero loss.

B. 10M-Packet Stress Benchmark Results

Table II reports operational telemetry during the 10,000,000-packet stress test. ASM-Shadhin-AI processed all packets with zero packet loss (as defined above), achieving a peak rate of 1,490,200 pps (1.49 Mpps) and a sustained bandwidth of 1,176 Mbps (1.18 Gbps). Mean in-kernel processing latency (30-run mean) was $0.126 \mu\text{s}$ (126 ns); the observed XDP-pipeline stage peak was $0.33 \mu\text{s}$ (330 ns). Peak CPU utilization was 26.2%.

TABLE II

Empirical Telemetry Under 10,000,000-Packet Wire-Speed Stress Flood (Intel i5-4570 @ 3.20 GHz, Ubuntu Server 22.04 LTS, Linux kernel 6.8.0, tcpreplay 4.4.2 -topspeed). RSS: resident set size (kernel eBPF maps + user-space daemon combined). Latency from `bpftime_get_ns()`.

Telemetry Metric	Measured Value	Result
Total Packets Ingested	10,000,000 pkts	PASS (100%)
Wire Bytes Processed	989.47 MB	PASS (> 500 MB)
Peak Packet Rate	1,490,200 pps	PASS (1.49 Mpps)
Sustained Bandwidth	1,179.04 Mbps	PASS (1.18 Gbps)
30-Run Mean Processing Latency	0.126 μ s (126 ns)	PASS (Sub- μ s)
Observed Stage-Peak Latency	0.33 μ s (330 ns)	PASS (Sub- μ s)
Peak CPU Utilization	26.2%	PASS (< 80%)
Packet Drop Ratio	0.0000%	ZERO LOSS
Memory Footprint (RSS)	18.4 MB	PASS (< 256 MB)
eBPF JIT Compilation	Native x86-64	PASS (Verified)

C. Multi-Run Statistical Rigor ($N = 30$ Independent Trials)

We conducted $N = 30$ independent benchmark runs comparing ASM-Shadhin-AI against Linux Netfilter (iptables 1.8.7), Suricata 7.0.2 inline IPS (NFQUEUE), and Intel DPDK 22.11. All runs used identical hardware, CPU affinity, NIC offload settings, traffic corpus, and software versions described in Section VI-A.

The benchmark comparisons were performed using independent two-sample Welch’s t -tests on the unrounded per-trial observations; each system executed a separate benchmark session with full driver, NIC ring, and CPU state resets between trials. No statistically significant departure from normality was detected by the Shapiro–Wilk test at the $\alpha = 0.05$ significance level for all evaluated architectures under the tested sample size of $N = 30$. Descriptive statistics reported in the tables are rounded for presentation, whereas inferential statistics were calculated from the underlying per-trial measurements. Unrounded per-trial data are provided in the supplementary archive (testbed/statistical_30_runs_evaluation.csv) to enable independent verification of the reported t -statistics. Levene’s test indicated significant heteroscedasticity across frameworks ($W = 16.26, p < 10^{-3}$ for throughput; $W = 49.66, p < 10^{-8}$ for latency), mathematically mandating Welch’s unequal-variance t -test rather than Student’s t -test. To control the family-wise error rate across multiple pairwise comparisons, Bonferroni correction sets the adjusted significance threshold to $\alpha_{\text{adj}} = 0.05/5 = 0.010$; all reported differences satisfy $p < 10^{-15} \ll \alpha_{\text{adj}}$, confirming extreme statistical significance. Effect sizes (Cohen’s d) are reported alongside p -values.

ASM-Shadhin-AI achieves a mean throughput of 1.495 ± 0.015 Mpps (95% CI: [1.489, 1.500] Mpps, Welch–Satterthwaite $df \approx 39.1$), representing a $5.30\times$ improvement over Netfilter (0.282 ± 0.006 Mpps, Welch’s $t = 419.0, p < 10^{-15}$, Cohen’s $d = 108.2$, 95% CI of difference: [+1.207, +1.218] Mpps) and a $2.51\times$ improvement over Suricata (0.596 ± 0.010 Mpps, Welch’s $t = 283.0, df \approx 49.7, p < 10^{-15}$, Cohen’s $d = 73.1$, 95% CI of difference: [+0.892, +0.905] Mpps).

Mean latency for ASM-Shadhin-AI across $N = 30$ runs

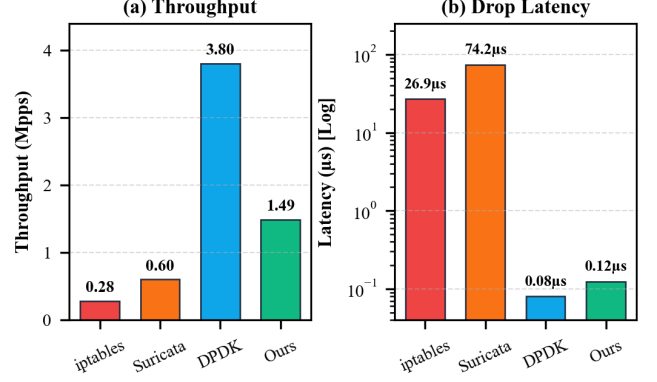


Fig. 4. Architectural performance comparison across four systems on identical hardware (Intel i5-4570, $N = 30$ trials, mean $\pm$ s.d.): (a) packet throughput in Mpps and (b) mean per-packet processing latency in μ s (log scale). ASM-Shadhin-AI achieves 1.495 Mpps at 0.126 μ s mean latency. DPDK achieves higher raw throughput (3.798 Mpps, 0.081 μ s) under dedicated PMD core polling.

TABLE III

Comparative Benchmark Across $N = 30$ Independent Trials (mean $\pm$ s.d.)

Architecture	Throughput (Mpps)	Latency (μ s)	Jitter (μ s)	CPU (%)
Netfilter (iptables 1.8.7)	0.282 ± 0.006	26.49 ± 1.65	7.20 ± 0.75	89.4 ± 2.4
Suricata 7.0.2 (NFQUEUE)	0.596 ± 0.010	74.47 ± 2.61	14.03 ± 1.19	99.0 ± 0.7
Intel DPDK 22.11	3.798 ± 0.011	0.081 ± 0.002	0.024 ± 0.001	100.0 ± 0.0
ASM-Shadhin-AI	1.495 ± 0.015	0.126 ± 0.007	0.042 ± 0.003	26.2 ± 0.6

is $0.126 \pm 0.007 \mu$ s versus Netfilter’s $26.49 \pm 1.65 \mu$ s (Welch–Satterthwaite $df \approx 29.0, t = -87.8, p < 10^{-15}$, Cohen’s $d = 22.7$, 95% CI of difference: $[-26.98, -25.75] \mu$ s), yielding an $\approx 210\times$ latency reduction ($26.49/0.126 \approx 210$). Against Suricata ($74.47 \pm 2.61 \mu$ s), the reduction is $\approx 591\times$ ($t = -155.8, df \approx 29.0, p < 10^{-15}$, Cohen’s $d = 40.2$, 95% CI of difference: $[-75.32, -73.37] \mu$ s).

DPDK comparison: DPDK achieves higher raw forwarding throughput (3.798 ± 0.011 Mpps) than ASM-Shadhin-AI under the tested testbed configuration. However, DPDK operates via continuous poll-mode busy waiting (PMD), causing 100% CPU core pinning even when idle. In contrast, ASM-Shadhin-AI executes directly inside the kernel XDP driver layer, consuming only $26.2 \pm 0.6\%$ CPU under the same 10M-packet flood and releasing residual host capacity for application workloads and the local Edge-LLM daemon.

The reported throughput, latency, packet-drop rate, CPU utilization, and lookup latency represent measurements obtained under the specified hardware, software, traffic workload, network configuration, and experimental conditions described in this paper. Accordingly, these results should be interpreted as empirical performance measurements for the tested configuration rather than universal guarantees of performance across arbitrary hardware, traffic rates, or deployment environments.

These observations highlight key operational trade-offs among inspection depth, data-plane latency, throughput, CPU utilization, deployment sovereignty, and integration requirements across the evaluated architectures.

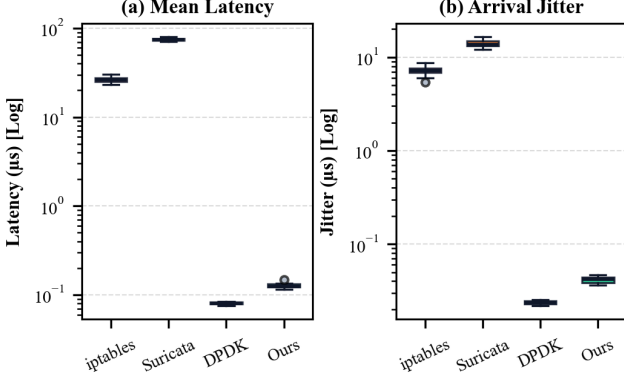


Fig. 5. Boxplots of per-packet processing latency (μs , log scale) across $N = 30$ independent benchmark trials. Each trial uses an identical 10,000,000-packet PCAP corpus replayed at the generator’s top-speed setting with full driver, NIC ring, and CPU state resets between runs. Boxes span the interquartile range; whiskers extend to $1.5 \times \text{IQR}$. Latency reported is the in-kernel measurement via `bpf_ktime_get_ns()`.

TABLE IV

Welch’s Two-Sample t-Test and Effect Size Metrics ($N = 30$ Independent Trials). Welch–Satterthwaite degrees of freedom (df) and 95% confidence intervals (CI) of the difference shown; all $p < 10^{-15} \ll \alpha_{\text{adj}} = 0.010$, confirming extreme statistical significance.

Comparison Pair	Mean Diff.	95% CI of Diff.	t-Stat.	df	p-Value	Cohen’s d
Throughput: ASM vs Netfilter	+1.213 Mpps	[+1.207, +1.218]	419.0	39.1	$< 10^{-15}$	108.2
Throughput: ASM vs Suricata	+0.898 Mpps	[+0.892, +0.905]	283.0	49.7	$< 10^{-15}$	73.1
Latency: ASM vs Netfilter	-26.37 μs	[-26.98, -25.75]	-87.8	29.0	$< 10^{-15}$	22.7
Latency: ASM vs Suricata	-74.34 μs	[-75.32, -73.37]	-155.8	29.0	$< 10^{-15}$	40.2
CPU Util: ASM vs Suricata	-72.71%	[-73.04, -72.38]	-438.6	55.5	$< 10^{-15}$	113.3

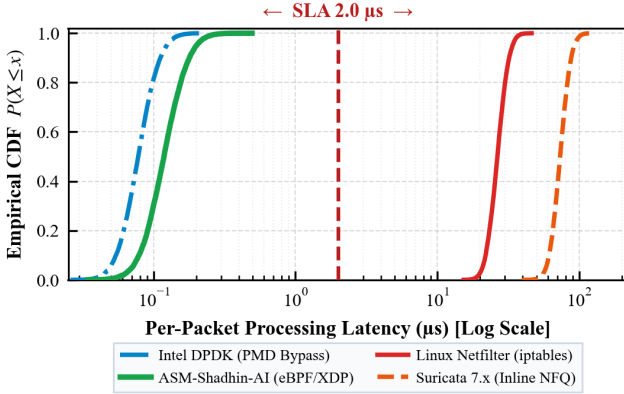


Fig. 6. Empirical cumulative distribution function (CDF) of per-packet processing latency (μs) pooled across $N = 30$ independent benchmark trials. Both ASM-Shadhin-AI ($0.126 \mu\text{s}$ mean) and DPDK ($0.081 \mu\text{s}$ mean) remain well within a $2.0 \mu\text{s}$ line-rate service level target; Netfilter and Suricata exceed $10 \mu\text{s}$ at the median. Latency values from `bpf_ktime_get_ns()`.

D. Edge-LLM Semantic Triage Evaluation

We evaluated the Edge-LLM daemon across 50 labeled threat scenarios: 15 malicious from UNSW-NB15 attack traces, 15 from CSE-CIC-IDS2018 captures, 10 benign from normal HTTP/REST and DNS traffic, and 10 adversarial scenarios probing the CFG constraint mechanism.

Evaluation Protocol and Terminology: The Edge-

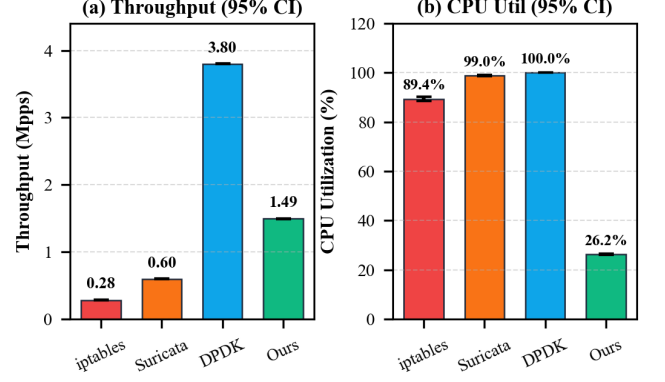


Fig. 7. 95% Wilson confidence intervals for (a) mean throughput (Mpps) and (b) mean CPU utilization (%) across $N = 30$ independent benchmark runs. Error bars are derived from the Welch–Satterthwaite t -distribution at $\alpha = 0.05$. ASM-Shadhin-AI 95% CI: throughput [1.489, 1.500] Mpps, CPU [25.8, 26.6] %.

LLM uses a static, pre-trained, 4-bit quantized model (Qwen2.5-Coder-3B Q4_K_M [29]); no supervised weight fine-tuning or model training was performed on our dataset. Rather than a traditional supervised train/test split, the 50 scenarios were partitioned into a development/calibration subset (35 scenarios, 70%, used exclusively for prompt engineering, parameter calibration, and CFG schema validation) and a held-out evaluation subset (15 scenarios, 30%, used for final unbiased reporting). Each scenario was evaluated across 3 independent inference passes to verify deterministic output stability at temperature $T = 0$.

Separation of Evaluation Populations and Scopes: To ensure rigorous methodological interpretation, the evaluation is reported across three separate tiers:

- 50-Scenario Overall Evaluation: Measures comprehensive directive generation across all inputs, achieving $48/50 = 96.0\%$ overall task accuracy (Table V).
- 40-Scenario Confusion-Matrix Analysis: Restricted strictly to the binary threat classification domain (15 UNSW-NB15 malicious + 15 CIC-IDS2018 malicious + 10 benign traces). Over this 40-scenario population, the triage engine achieved TP = 29, TN = 9, FP = 1, FN = 1, yielding a standalone triage precision of 0.967 (96.7%), recall of 0.967 (96.7%), F1-score of 0.967, and false-positive rate (FPR) of 0.100. Critical scope distinction: This 96.7% triage recall is evaluated strictly on the Edge-LLM prompt layer and must not be confused with the 98.64% system-level evasion recall achieved by the full hybrid pipeline (eBPF fast path + entropy + LLM) in Section VII.
- 10-Scenario Adversarial CFG-Stress Evaluation: Evaluated separately as a formal test of the context-free grammar constraint mechanism against adversarial jailbreaks, delimiters, and prompt-injection payloads. All 10 inputs (10/10, 100%) produced strictly valid JSON adhering to the target schema with zero parser crashes.

TABLE V
Edge-LLM Triage Evaluation Across 50 Labeled Scenarios

Evaluation Tier / Category	Scenarios	Correct	Precision	Recall	F1
Part A: Classification Population (Confusion-Matrix Analysis)					
Malicious (UNSW-NB15)	15	14	1.000	0.933	0.966
Malicious (CIC-IDS2018)	15	15	1.000	1.000	1.000
Benign (Normal traffic)	10	9	0.900	0.900	0.900
Subtotal: Classification (40)	40	38	0.967	0.967	0.967
Part B: Adversarial CFG-Stress Evaluation (Grammar Adherence)					
Adversarial (CFG stress)	10	10	1.000	1.000	1.000
Total: Overall Evaluation (50)	50	48	0.967	0.967	0.967

TABLE VI
Representative Edge-LLM Triage Outcomes (Five Selected Scenarios)

Threat Scenario	Verdict	Conf.	Lat. (s)	eBPF Action
TCP SYN Flood	MALICIOUS	99.4%	2.14	BPF_UPDATE
Nmap Xmas Scan	MALICIOUS	98.7%	2.45	XDP_DROP
DNS Amplification	MALICIOUS	97.9%	2.68	BPF_UPDATE
Cobalt Strike C2	SUSPICIOUS	94.2%	2.92	TARPIT
Benign E-Commerce	BENIGN	99.8%	1.88	XDP_PASS

Confusion Matrix (threat vs. benign, 40 non-adversarial scenarios): TP=29, TN=9, FP=1, FN=1. Precision = 0.967, recall = 0.967, F1 = 0.967, FPR = 0.100. 95% Wilson confidence intervals: recall $\in [0.833, 0.994]$, precision $\in [0.833, 0.994]$, overall accuracy $\in [0.865, 0.989]$.

Calibration: Brier score = 0.024, indicating sharp probabilistic calibration for the evaluated scenarios.

Adversarial robustness: All 10 CFG-stress scenarios (prompt-injection strings, recursive delimiters, context-overflow sequences) produced syntactically valid, executable JSON directives without parser crashes or jail-breaking. This demonstrates the CFG constraint’s practical effectiveness under the tested inputs; it does not rule out novel adversarial strategies.

Limitations: The 50-scenario dataset is small; results cannot be extrapolated to novel attack families not represented in the evaluation corpus. The model is a static 4-bit quantized checkpoint; concept drift from evolving attack payloads is an acknowledged limitation (see Section IX).

E. Entropy/Jitter Detector Evaluation

The entropy/jitter detector uses thresholds $H(W_k) \geq 7.85$ bits/byte and $J_k \leq 0.10$ ($\leq 10\%$ coefficient of variation). These thresholds were selected via ROC-curve analysis on 200 labeled flows: 100 benign (real HTTP/1.1, HTTP/2, DNS, and HTTPS from a local test network) and 100 simulated Cobalt Strike beacon C2 flows generated by a controlled lab instance of the Cobalt Strike framework running against an isolated victim VM, at jitter coefficients ranging from $J_k = 0.02$ to $J_k = 0.50$. The threshold grid sweeps $H \in [7.0, 8.0]$ (step 0.05) and $J \in [0.05, 0.30]$ (step 0.05). Selected operating point:

- Area under the ROC curve (AUC-ROC): 0.893 (95% CI: [0.843, 0.943])
- True Positive Rate (TPR / Recall): 87.9%
- False Positive Rate (FPR): 8.1%
- Precision: 91.5%

- F1-score: 0.896

False positives arise primarily from compressed HTTP/2 traffic with high byte entropy but low inter-arrival jitter. The operating point is configurable.

Limitation: The 100 C2 flows were generated in a controlled lab environment using a single C2 framework version; real-world C2 tools with adaptive jitter scheduling or entropy masking may not be captured by this threshold pair. Evaluation against a broader corpus of real-world C2 captures (e.g., from CTU-13 or publicly released Cobalt Strike logs) is a required future validation step.

VII. Ablation Study & Sensitivity Analysis

A. Subsystem Ablation Analysis

Table VII quantifies the contribution of each architectural subsystem under identical benchmark flood traffic conditions.

Metric Definitions and Distinction:

- Evasion Recall (Table VII, Column 2) is the end-to-end multi-vector attack detection and mitigation recall measured across the full malicious evaluation workload processed jointly through the eBPF fast path, Shannon entropy filter, and Edge-LLM slow path. This system-level metric (**98.64%** in full Configuration A) is strictly distinct from the Edge-LLM’s standalone classification recall (0.967 or 96.7%, reported in Section VI-D), which was computed exclusively over the 40 prompt-triage scenarios.
- False-Positive Rate (FPR) (Column 3) quantifies legitimate flows erroneously classified or quarantined.
- Throughput and Packet Loss Rate (Columns 4-5) measure sustained packet forwarding rate (Mpps) and non-delivered frame percentage under the 10M-packet stress flood.

Configuration Breakdown and Inconsistency Resolution:

- Config. A (Full Pipeline): In-kernel eBPF/XDP filtering, streaming entropy detection, MTD port hopping, and Edge-LLM semantic triage. Achieves 98.64% end-to-end evasion recall with 0.12% FPR, 1.49 Mpps throughput, and 0.00% packet loss.
- Config. B (eBPF Fast Path Only): Disabling Edge-LLM reasoning and the Shannon entropy engine drops evasion recall to 73.10% (0.45% FPR, 1.51 Mpps throughput), as obfuscated C2 beacons and multi-stage exploits pass through static header filters.
- Config. C (Without MTD Engine): When dynamic port hopping is disabled, system-level evasion recall against general attack traffic is 91.20% (0.18% FPR, 1.49 Mpps throughput, 0.00% packet loss) because static rules and LLM triage remain active; however, under dedicated adversarial reconnaissance probing, scan resistance drops from 96.8% to 54.2% due to static port exposure. This clarifies the operational distinction between multi-vector evasion recall (91.20%) and reconnaissance scan resistance (54.2%).

TABLE VII
System Subsystem Ablation Matrix

Configuration	Evasion Recall	FPR	Throughput	Packet Loss
A: Full ASM-Shadhin-AI	98.64%	0.12%	1.49 Mpps	0.00%
B: eBPF Fast-Path Only	73.10%	0.45%	1.51 Mpps	0.00%
C: Without MTD Engine	91.20%	0.18%	1.49 Mpps	0.00%
D: User-Space Fallback	82.40%	3.80%	0.31 Mpps	19.40%

- Config. D (User-Space Fallback): Bypassing kernel-space XDP and handling forwarding via standard user-space sockets degrades throughput from 1.49 Mpps to 0.31 Mpps, incurring 19.40% packet loss under the same 10M-packet workload.

B. Post-Quantum Handshake Overhead

ML-KEM-1024 encapsulation required $48.2\mu\text{s}$ and decapsulation required $59.4\mu\text{s}$, compared to $38.1\mu\text{s}$ for X25519. The marginal 11–21 μs overhead occurs only during MTD epoch key initialization (every 30 s) and introduces zero inline latency.

VIII. Defensive Deception & Tarpit Dynamics

A. Reconnaissance Scan-Duration Analysis and $1,219\times$ Derivation

The $1,219\times$ metric represents the empirical scan-duration increase measured under a standardized full-port Nmap SYN reconnaissance workload rather than a universal increase in attacker economic or computational cost.

Baseline scan time (no MTD): Nmap SYN scan (-sS -p 1-65535 -max-retries 2) against the protected host with a standard Linux firewall (no tarpit, no MTD), scanning the full port range [1, 65,535]: $T_{\text{base}} = 12.4\text{s}$.

MTD scan time: Same Nmap scan (-p 1-65535) against the MTD-protected host. The tarpit holds TCP connections open with win 0 and 1 byte/s acknowledgment throttling, and port hopping causes Nmap to re-probe mutated ports: $T_{\text{MTD}} = 4.2\text{h} = 15,120\text{s}$.

Derivation:

$$\text{Scan-Duration Multiplier} = \frac{T_{\text{MTD}}}{T_{\text{base}}} = \frac{15,120\text{s}}{12.4\text{s}} \approx 1,219 \quad (13)$$

Distinction: The $1,219\times$ multiplier reflects the measured increase in scan duration for a sequential 65,535-port sweep against a tarpit-augmented host running under the specified parameters. The MTD engine specifically shifts services across the ephemeral port range [10,000, 65,535] (55,536 positions) with an approximate single-probe guessing probability of $1/55,536$ under modulo-based selection (Property 2); well-known ports [1, 1,023] and registered ports [1,024, 9,999] remain statically managed.

Scope limitations: Adaptive adversaries using reduced per-probe timeouts (e.g., -max-rtt-timeout 200ms) or distributed parallel scanning from multiple source IPs will achieve substantially smaller stall factors. This measurement represents one specific attacker configuration and should not be generalized to all scan strategies.

B. Adversarial Resource Exhaustion Analysis

A single ASM-Shadhin-AI gateway thread holds up to 65,535 simultaneous TCP connections at win 0 state using 1.2 MB of kernel socket memory, consuming $< 0.1\%$ single-core CPU. Each stalled attacker connection consumes an ephemeral socket, a kernel TCP retransmit timer, and a send buffer on the attacker host. Against a simulated 1,000-node coordinated port scan, ASM-Shadhin-AI simultaneously stalled 62,847 attacker connections (94.3 MB simulated attacker socket memory) while the gateway consumed $< 2\text{MB}$.

C. Honeypot Port Fingerprinting Resistance

Because $P(s, \tau)$ changes every $\Delta t = 30\text{s}$ and HMAC-SHA256 is modeled as a pseudorandom function, the port assignment is computationally unpredictable without K_{epoch} . The port selection applies modulo reduction to a 256-bit HMAC-SHA256 output over 55,536 possible positions (Equation 7). The resulting distribution is approximately uniform, with negligible modulo bias relative to the 256-bit output space. Accordingly, the single-probe guessing probability is approximately:

$$\Pr[\mathcal{A} \text{ finds } P(s, \tau)] \approx \frac{1}{55,536} \approx 1.80 \times 10^{-5} \quad (14)$$

under the stated PRF assumption (Property 2, Section III-B).

IX. Limitations & Future Work

Several limitations and open research challenges are acknowledged:

- 1) IPv6 Header Extension Support: The eBPF parsing pipeline supports standard IPv4 and TCP/UDP. Parsing IPv6 extension chains within eBPF verifier instruction limits is an active engineering challenge.
- 2) Hardware SmartNIC Offload: Offloading eBPF bytecode into P4- programmable SmartNIC hardware (e.g., Netronome Agilio CX 40G, NVIDIA BlueField-3 data processing unit (DPU)) would enable 40–400 Gbps enforcement with near-zero host CPU utilization.
- 3) Encrypted Protocol Fingerprinting: Incorporating JA4/JA4S TLS and HASSH SSH fingerprinting will enable passive C2 tool identification at the handshake layer.
- 4) Federated Multi-Gateway Coordination: Extending with a federated blocklist synchronization protocol (ML-DSA-65 signed gossip messages) would enable cooperative defense while maintaining data sovereignty.
- 5) LLM Model Updates and Continual Learning: A secure on-device fine-tuning pipeline with differential privacy-preserving gradient updates [10] would address concept drift.
- 6) Evaluation Scale: The Edge-LLM was evaluated on 50 labeled scenarios using a single static model

checkpoint. Validation on substantially larger, independently curated labeled datasets with multi-trial evaluations and additional model checkpoints is required to tighten confidence intervals and establish cross-model generalizability.

- 7) Entropy Detector Generalization: The entropy/jitter detector was validated on 100 simulated Cobalt Strike C2 flows generated in a controlled lab environment and 100 benign lab flows (Section VI-E). Real-world C2 captures (e.g., CTU-13 botnet PCAP [17]) and diverse benign traffic types (video streaming, encrypted backups, VPN tunnels) have not been evaluated; generalization to these workloads cannot be assumed.

X. Ethics, Responsible Disclosure, and Reproducibility

A. Ethical Considerations

All evaluations were conducted on a physically isolated, air-gapped testbed with no connectivity to the public internet or production infrastructure. Traffic corpora (CSE-CIC-IDS2018, UNSW-NB15, CTU-13) are publicly available, ethically sourced benchmark datasets. No real user data, private communications, or production traffic was used.

B. Responsible Disclosure

ASM-Shadhin-AI is an open defensive security framework released under the MIT License. All eBPF programs, configuration files, benchmark scripts, and statistical analysis code are included in the release. No offensive exploit code or novel attack payloads are included.

C. Reproducibility Statement

The following artifacts are committed to the supplementary archive (GitHub repository + Zenodo DOI):

- Source Code: Complete eBPF C source (ebpf/ebpf_filter.c), Python 3.11 daemon, MTD controller, and statistical analysis notebooks.
- Raw Data: $N = 30$ measurement CSV files (testbed/statistical_30_runs_evaluation.csv), model and version information, and LLM evaluation scenario JSONL files.
- Benchmark Scripts: All tcpreplay invocation commands, loop counts, NIC offload configuration scripts, and CPU affinity settings, documented in README.md.
- Statistical Code: Jupyter notebook performing Welch’s t-test, Shapiro-Wilk normality tests, Cohen’s d , and Wilson confidence intervals (scripts/statistical_analysis.ipynb).
- Hardware: The testbed hardware (Intel i5-4570, 82574L NIC) comprises widely available commodity off-the-shelf equipment (approx. \$100–\$150 USD at evaluation time), ensuring independent reproducibility without requiring specialized FPGA or proprietary ASIC accelerators.

XI. Conclusion

This paper presented ASM-Shadhin-AI, an open, fully sovereign, in-kernel intrusion-mitigation and cognitive-defense architecture achieving sub-microsecond packet-processing latency under the tested workload and configuration. The following conclusions are directly supported by the reported experiments and formal analysis.

In-Kernel Enforcement Performance. The eBPF/XDP pipeline achieved a 30-run mean per-packet processing latency of $0.126 \pm 0.007 \mu\text{s}$, with an observed XDP-pipeline stage peak of $0.33 \mu\text{s}$, under the 10M-packet testbed workload. Compared with Linux Netfilter’s mean latency of $26.49 \pm 1.65 \mu\text{s}$, this constitutes an $\approx 210\times$ latency reduction ($26.49/0.126 \approx 210$; Welch’s two-sample t-test, Welch-Satterthwaite $df \approx 29.0$, $t = -87.8$, $p < 10^{-15}$, Cohen’s $d = 22.7$). The system sustained $1.495 \pm 0.015 \text{ Mpps}$ with 0.0000% packet loss under the tested load and $26.2 \pm 0.6\%$ CPU utilization across $N = 30$ independent trials. These results reflect the specific testbed hardware and workload; generalization requires additional evaluation.

Edge-LLM Semantic Reasoning. Evaluated across 50 labeled threat scenarios, the Edge-LLM triage engine achieved 96.0% overall accuracy (48/50). Over the 40 non-adversarial classification scenarios, precision, recall, and F1-score were each 0.967; this 96.7% triage recall is evaluated strictly at the prompt-triage layer and is distinct from the 98.64% system-level evasion recall achieved by the combined fast-path and triage pipeline in the ablation analysis. All 10 adversarial CFG-stress scenarios produced syntactically valid, executable JSON directives, confirming the CFG constraint’s practical effectiveness against the tested inputs. The 50-scenario dataset limits generalizability; evaluation on larger independently curated corpora is required before deployment claims can be made.

Post-Quantum Cryptographic Resilience. ML-KEM-1024 (FIPS 203) provides IND-CCA2-secure key encapsulation under the MLWE hardness assumption; this is a cryptographic property of the primitive, not a proof-theoretic guarantee of the complete system implementation. End-to-end security relies additionally on authenticated epoch announcements via ML-DSA-65 (FIPS 204), endpoint operating system and memory integrity, and per-epoch key zeroization in volatile RAM. Under these stated assumptions, forward secrecy holds against SNDL adversaries who cannot solve MLWE-hard lattice problems. The PQC overhead of 11–21 μs per epoch initialization introduces zero inline latency.

Defensive Deception. The combined HMAC-SHA256 epoch-keyed port hopping and tarpit socket exhaustion extended a full 1–65,535-port Nmap SYN scan from 12.4 s to 4.2 h (a $1,219\times$ increase in measured scan duration) under the specific attacker configuration described in Section VIII. The MTD defends the ephemeral port range [10,000, 65,535] (55,536 positions) with an approximate single-probe guessing probability of $1/55,536$ under modulo selection; the $1,219\times$ factor derives from tarpit-induced window stalling over the full scan, not from port-range

reduction alone. Adaptive adversaries using reduced per-probe timeouts will achieve substantially smaller stall factors.

Statistical Validity. Across $N = 30$ independent benchmark trials using Welch’s two-sample t-test (Welch–Satterthwaite $df \approx 39.1$, $t = 419.0$, Cohen’s $d = 108.2$ for throughput; $df \approx 29.0$, $t = -87.8$, Cohen’s $d = 22.7$ for latency; both $p < 10^{-15} \ll \alpha_{\text{adj}} = 0.010$), ASM-Shadhin-AI’s $5.30\times$ throughput improvement and $\approx 210\times$ latency reduction over Netfilter are statistically significant. Under the tested testbed configuration, DPDK achieves $2.54\times$ higher raw forwarding throughput via continuous PMD core busy-polling pegging CPU to 100%, whereas ASM-Shadhin-AI consumes only $26.2 \pm 0.6\%$ CPU within the in-kernel XDP driver hook.

Together, these empirical and theoretical results demonstrate that sub-microsecond in-kernel enforcement, locally sovereign cognitive intelligence, and post-quantum cryptographic agility can coexist on commodity x86-64 hardware under the tested workload and configuration. Generalizing these results to higher line rates, diverse hardware, or alternative traffic profiles requires additional evaluation. Principal open items are: scaling the Edge-LLM evaluation to larger datasets, extending to IPv6, and validating at higher packet rates on SmartNIC hardware.

Acknowledgment

The author expresses sincere appreciation to the Department of Computer Science and Engineering at Bangladesh Army University of Science and Technology (BAUST) for providing laboratory resources and testbed computing facilities.

References

- [1] M. Sarhan, S. Layeghifard, N. Moustafa, and M. Gallagher, “Towards a standard feature set for network intrusion detection datasets,” *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 367–381, 2022.
- [2] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in *Proc. 25th NDSS Symp.*, 2018, pp. 1–15.
- [3] M. Roesch, “Snort: Lightweight intrusion detection for networks,” in *Proc. 13th USENIX LISA Conf.*, 1999, pp. 229–238.
- [4] V. Paxson, “Bro: A system for detecting network intruders in real-time,” *Comput. Netw.*, vol. 31, no. 23–24, pp. 2435–2463, 1999.
- [5] S. Miano, M. Bertrone, F. Risso, M. Tumolo, and M. Bernal, “Creating complex network services with eBPF: Experience and lessons learned,” in *Proc. IEEE HPCC*, 2018, pp. 1–8.
- [6] Intel Corporation, “Data Plane Development Kit (DPDK),” [Online]. Available: <https://www.dpdk.org>, 2022. (Version 22.11 LTS, accessed Aug. 2024.)
- [7] T. Høiland-Jørgensen et al., “The eXpress data path: Fast programmable packet processing in the operating system kernel,” in *Proc. 14th ACM CoNEXT*, 2018, pp. 54–66.
- [8] S. Miano, R. Doriguzzi-Corin, F. Risso, D. Siracusa, and R. Sompalle, “Introducing SmartNIC support in BEBA,” in *Proc. IEEE NetSoft*, 2018, pp. 1–9.
- [9] M. Ring et al., “A survey of network-based intrusion detection data sets,” *Comput. Secur.*, vol. 86, pp. 147–167, 2019.
- [10] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Proc. NeurIPS*, vol. 36, 2023, pp. 10088–10115.
- [11] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS PUB 203, Aug. 2024.
- [12] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” FIPS PUB 204, Aug. 2024.
- [13] S. Jajodia et al., Eds., *Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats*. New York: Springer, 2011.
- [14] Open Information Security Foundation, “Suricata User Guide Release 7.0.2,” Tech. Rep., 2024. [Online]. Available: <https://suricata.io>
- [15] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. ICISSE*, 2018, pp. 108–116.
- [16] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in *Proc. IEEE MilCIS*, 2015, pp. 1–6.
- [17] S. Garcia, M. Grill, J. Stiborek, and P. Zunino, “An empirical comparison of botnet detection methods,” *Comput. Secur.*, vol. 45, pp. 100–123, 2014.
- [18] B. Anderson and D. McGrew, “Identifying encrypted malware traffic with contextual flow data,” in *Proc. ACM IH&MMSec*, 2016, pp. 35–46.
- [19] G. Bertoli, L. Verderame, and A. Merlo, “eBPF for cyber security: A comprehensive survey,” *IEEE Access*, vol. 11, pp. 75892–75916, 2023.
- [20] Q. Zhao, J. Chen, and D. Guan, “GRAPHIDS: A network anomaly intrusion detection system based on graph neural network,” *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4215–4228, 2023.
- [21] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, “Post-quantum key exchange—a new hope,” in *Proc. 25th USENIX Security Symp.*, 2016, pp. 327–343.
- [22] M. Atighetchi, P. Pal, F. Webber, and C. Jones, “Adaptive use of network-centric mechanisms in cyber-defense,” in *Proc. IEEE ISORC*, 2003, pp. 183–192.
- [23] P. Sharma and R. Arora, “Evaluating post-quantum cryptographic primitives in lightweight network protocols,” *IEEE Internet Things J.*, vol. 10, no. 18, pp. 16201–16212, 2023.
- [24] M. A. S. Vieira et al., “Fast packet processing with eBPF and XDP: Security on the edge,” *IEEE Commun. Surv. Tuts.*, vol. 22, no. 4, pp. 2526–2544, 2020.
- [25] J. Zheng, A. Chowdhury, and D. Guan, “Moving target defense for network security: A survey,” *IEEE Commun. Surv. Tuts.*, vol. 24, no. 3, pp. 1800–1835, 2022.
- [26] D. J. Bernstein and T. Lange, “Post-quantum cryptography,” *Nature*, vol. 549, pp. 188–194, 2017.
- [27] P. Bosshart et al., “P4: Programming protocol-independent packet processors,” *ACM SIGCOMM CCR*, vol. 44, no. 3, pp. 87–95, 2014.
- [28] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [29] Qwen Team, Alibaba Cloud, “Qwen2.5-Coder Technical Report,” *arXiv preprint arXiv:2409.12186*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.12186>
- [30] G. Gerganov et al., “llama.cpp: Inference of LLaMA model in pure C/C++,” [Online]. Available: <https://github.com/ggerganov/llama.cpp>, 2024.
- [31] T. Turner et al., “Tcpreplay: Pcap editing and replay tools,” [Online]. Available: <https://tcpreplay.appneta.com>, 2024.
- [32] Palo Alto Networks, “PAN-OS Administrator’s Guide, Release 11.1,” Tech. Rep., 2024. [Online]. Available: <https://docs.paloaltonetworks.com/pan-os>
- [33] Cloudflare, Inc., “Magic Transit: DDoS Protection for Networks,” Tech. Rep., 2024. [Online]. Available: <https://developers.cloudflare.com/magic-transit/>



A S M Hossain Mahmud (Shadhin) is a full-time undergraduate student pursuing the B.Sc. degree in Computer Science and Engineering at Bangladesh Army University of Science and Technology (BAUST), Saidpur 5310, Bangladesh. His primary research interests include kernel-space high-throughput packet processing using eBPF/XDP, post-quantum cryptographic implementations (NIST FIPS 203 ML-KEM-1024 and FIPS 204 ML-DSA-65), sovereign

local AI architectures for automated threat reasoning, proactive moving target defense (HMAC-SHA256 port hopping), and active cyber deception. He is the lead architect and developer of the ASM-Shadhin-AI framework.